

Python Programming

Unit 04 – Lecture 05 Notes

DB-API 2.0, CRUD Operations, MongoDB Overview

Tofik Ali

February 20, 2026

Contents

1	Lecture Overview	1
2	Core Concepts	2
2.1	DB-API 2.0 Workflow (SQLite Example)	2
2.2	CRUD in SQL	2
2.3	Parameterised Queries	2
2.4	Fetching Results	2
2.5	MongoDB Overview	3
3	Demo Walkthrough	3
4	Interactive Checkpoints (with Solutions)	3
5	Practice Exercises (with Solutions)	3
6	Exit Question (with Solution)	4

1 Lecture Overview

Python connects to relational databases using the DB-API 2.0 pattern:

- open a connection,
- create a cursor,
- execute SQL queries,
- fetch results,
- commit changes (for INSERT/UPDATE/DELETE),
- close connection.

This lecture uses SQLite (`sqlite3`) for hands-on CRUD and discusses MongoDB at a high level.

2 Core Concepts

2.1 DB-API 2.0 Workflow (SQLite Example)

```
import sqlite3

con = sqlite3.connect("app.db")
cur = con.cursor()

cur.execute("CREATE TABLE IF NOT EXISTS students(id INTEGER PRIMARY KEY, name TEXT)")
con.commit()

cur.execute("SELECT * FROM students")
rows = cur.fetchall()

con.close()
```

2.2 CRUD in SQL

- **Create:** INSERT INTO table (...) VALUES (...)
- **Read:** SELECT ... FROM table WHERE ...
- **Update:** UPDATE table SET ... WHERE ...
- **Delete:** DELETE FROM table WHERE ...

2.3 Parameterised Queries

Never build SQL by concatenation with user input:

```
# BAD (do not do this)
cur.execute("SELECT * FROM students WHERE sapid = '" + sapid + "'")
```

Use placeholders:

```
cur.execute("SELECT * FROM students WHERE sapid = ?", (sapid,))
```

Benefits:

- avoids SQL injection,
- handles quoting automatically,
- reduces bugs.

2.4 Fetching Results

Common methods:

- `fetchone()` for a single row
- `fetchall()` for all rows

2.5 MongoDB Overview

MongoDB is a document database:

- data is stored as documents (JSON-like),
- documents are grouped into collections.

Python driver: `pymongo` (external install). Example (conceptual):

```
# pip install pymongo
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017")
db = client["college"]
students = db["students"]
students.insert_one({"name": "Asha", "sapid": "5001", "marks": 88})
```

3 Demo Walkthrough

SQLite CRUD: `demo/sqlite_crud_demo.py`

MongoDB (optional): `demo/mongodb_optional_demo.py`

4 Interactive Checkpoints (with Solutions)

Checkpoint 1 Solution

Question: purpose of `cursor.execute(...)`?

Answer: It sends an SQL statement (or command) to the database engine to be executed.

Checkpoint 2 Solution

Question: why parameterised queries are safer?

Answer: They separate SQL code from data values, preventing injection and handling escaping correctly.

5 Practice Exercises (with Solutions)

Exercise 1: Update Marks

Task: Update marks for a student by SAP ID.

Solution:

```
cur.execute("UPDATE students SET marks = ? WHERE sapid = ?", (marks, sapid))
con.commit()
```

Exercise 2: Delete a Student

Task: Delete a row by SAP ID.

Solution:

```
cur.execute("DELETE FROM students WHERE sapid = ?", (sapid,))  
con.commit()
```

6 Exit Question (with Solution)

Question: SQL query to update marks by SAP ID?

Answer (pattern):

```
UPDATE students SET marks = ? WHERE sapid = ?;
```

Appendix: Slide Deck Content (Reference)

The material below is a reference copy of the slide deck content. Exercise solutions are explained in the main notes where applicable.

Title Slide

Repository: <https://github.com/tali7c/Python-Programming>

Quick Links

[Core Concepts](#) [Demo](#) [Interactive](#) [Summary](#)

Agenda

- Core Concepts
- Demo
- Interactive
- Summary

Learning Outcomes

- Understand the DB-API 2.0 workflow (connection, cursor, execute)
- Perform CRUD operations using parameterised queries
- Fetch results using `fetchone` / `fetchall`
- Describe how MongoDB differs from relational databases

DB-API 2.0: The Common Pattern

```
import sqlite3
con = sqlite3.connect("app.db")
cur = con.cursor()
cur.execute("SELECT * FROM students")
rows = cur.fetchall()
con.close()
```

CRUD

- Create: INSERT
- Read: SELECT
- Update: UPDATE
- Delete: DELETE

Parameterised Queries (Important)

- Avoid string concatenation in SQL
- Use placeholders to prevent SQL injection and quoting bugs

```
cur.execute(  
    "INSERT INTO students(name, sapid) VALUES (?, ?)",  
    (name, sapid)  
)
```

MongoDB (High-Level View)

- MongoDB is a document database (NoSQL)
- Stores JSON-like documents in collections
- Useful when schema is flexible or data is nested
- Python driver: `pymongo` (requires installation)

Demo: SQLite CRUD + Optional MongoDB Stub

- SQLite CRUD: `demo/sqlite_crud_demo.py`
- MongoDB (optional): `demo/mongodb_optional_demo.py`

Checkpoint 1

Question: What is the purpose of `cursor.execute(...)`?

Checkpoint 2

Question: Why are parameterised queries safer than string concatenation?

Think-Pair-Share

Discuss:

- Which operations should be allowed in a student database app: Insert, Update, Delete? Why?

Key Takeaways

- DB-API uses connection + cursor + execute + fetch + commit
- CRUD maps to SQL commands (INSERT/SELECT/UPDATE/DELETE)
- Parameterised queries improve safety and correctness
- MongoDB stores documents (NoSQL) and is accessed via drivers like `pymongo`

Exit Question

Write one SQL query to update marks of a student with a given SAP ID.